UNIVERSITY OF BIRMINGHAM

Physical Memory Access, Without the Hardware

A software-only pipeline for scanning and patching raw system memory that is built from a single-process proof of concept to a full physical-RAM scanner, with no capture card or any other hardware required.

Muhammed Razvi · MSc Cyber Security· Supervised by Tom Chothia



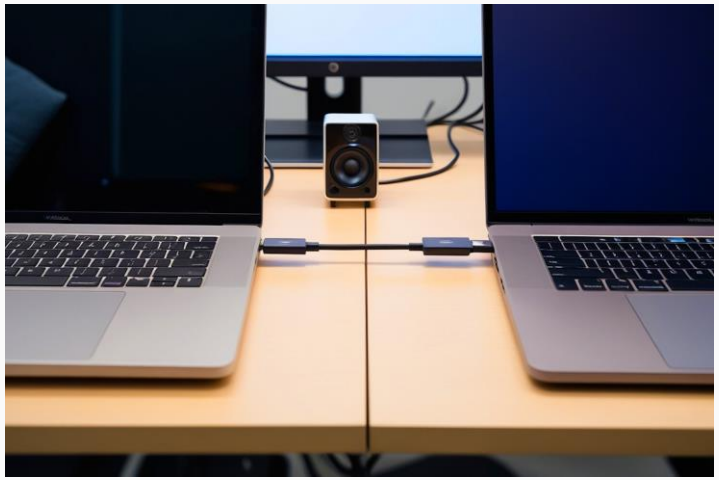
The Hardware Barrier

A DMA Capture Card



Purpose-built hardware, seated in a PCIe slot, reading RAM independent of the OS.

A Second PC



The capture card needs a separate host machine to receive the memory stream.

Physical Install



Case open, card seated, cabling run, before a single byte is ever read.

Where the Goal Moved

STARTED WITH

Live Process Scanning

Attach to a single running process, scanning its virtual memory, and locate a known byte, then narrow down the results, target script was built along with this as a proof of concept

LANDED ON

Physical Memory Access

The same machine ,scanning, searching and overwriting physical memory by scanning the entire raw physical memory image via the WinPmem driver API which is already signed by kernal of windows. no dedicated capture hardware, no second machine.

One Engine, Five Stages

01 **Live process scan** — Attach by PID, find a known byte pattern in a running process.

02 **Scan trace** — Walk memory region-by-region, showing reads, matches, and timing.

03 **Live scan + patch** — Overwrite matched bytes in a live process, with wildcard support.

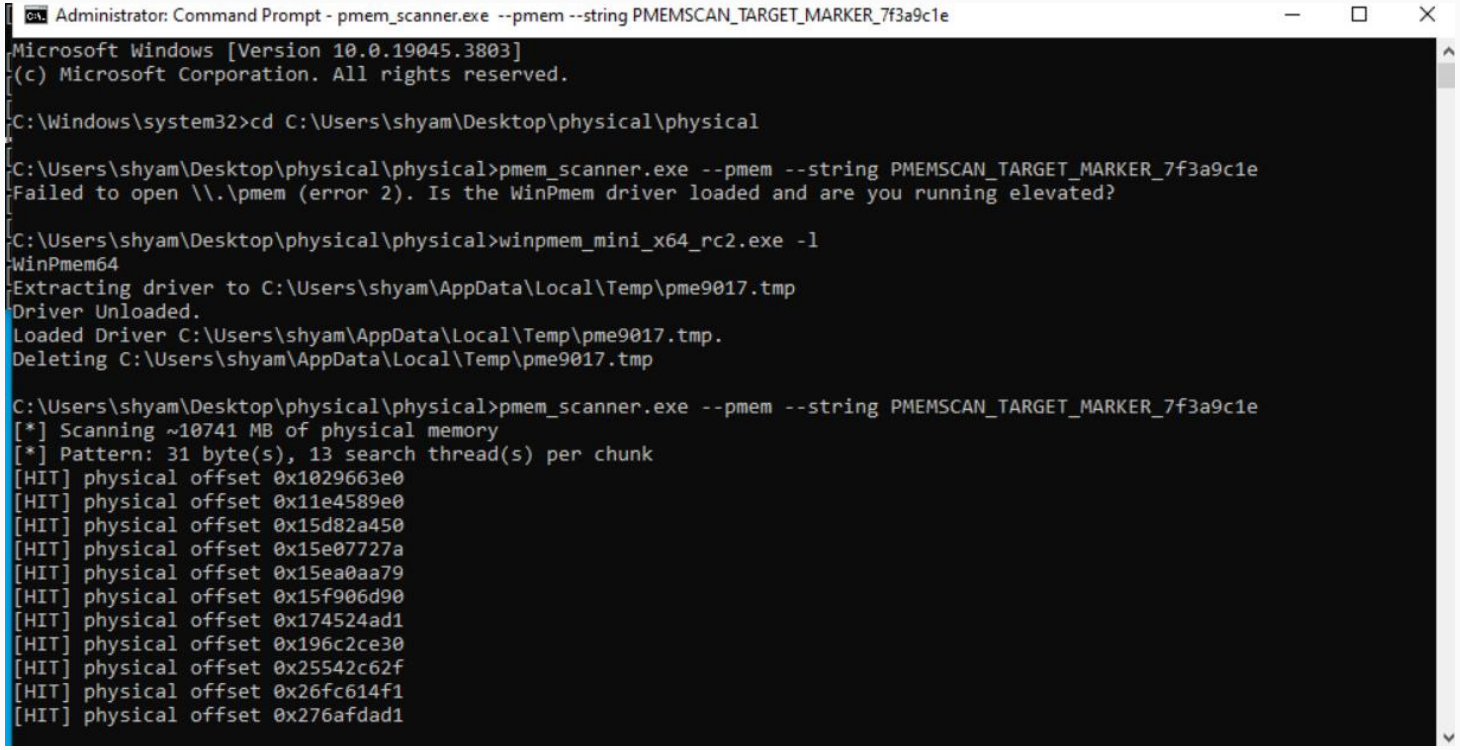
04 **Physical image scan** — Point the same engine at a raw RAM image instead of a live process.

05 **Physical image patch** — Patch a RAM image directly, with an automatic backup first.

The insight: *a process and a RAM image are both just a byte stream with a start offset , that's what let Stage 4 and 5 reuse Stages 1–3 unmodified.*

Proof: Finding a Live Process in Raw RAM

A known 31-byte marker was planted in a running process, then rediscovered independently by scanning the raw physical memory image without reference to the process, just the RAM.



~10.7 GB

physical memory scanned

13

parallel search threads

31 bytes

exact pattern matched

2m 05.08s

total scan time

Why It Matters

No hardware dependency

Runs on the target machine itself . nothing to buy, seat, or cable.

Reversible by design

Every patch operation writes a backup of the original bytes first.

General-purpose engine

Applies to forensics, VM introspection, and live debugging, not one fixed use case.

Lower cost, faster iteration

Every stage was tested and re-tested in minutes, not hardware setup cycles.

THE BIGGER PICTURE

Physical memory access is the same class of primitive behind recent VBS (Virtualization-Based Security) bypassing attacks like Download More RAM (USENIX '26, Chothia et al.) this project builds the underlying scan/patch engine that class of work depends on.

Goal Achieved.

From live-process scanning to a full physical-memory read, search, and patch pipeline is entirely in software.

KNOWN LIMITS, STATED PLAINLY

Offsets vs. addresses

This run was on a VirtualBox Windows 10 guest (10.7 GB, 13 vCPUs) offsets are guest-physical until validated on bare metal.

Elevated privileges

The first run failed with “Failed to open \\.pmem (error 2)” until WinPmem was loaded and the shell ran elevated.

Driver signing

The physical-memory driver path (WinPmem) depends on Windows' driver-signing policy on the host machine.

Thank you questions are welcomed.